Reinforcement Learning for Sim-to-Sim Robot Transfer

Leonardo Passafiume (358616) Lucio Baiocchi Maurizio Lanzoni Raiteri
Thomas Kelly

Abstract

This report studies reinforcement learning for continuous control and sim-to-real transfer in the simplified sim-to-sim setting proposed by the project. First, we implemented REINFORCE, REINFORCE with a constant baseline, and an actor-critic policy-gradient variant from scratch on `Hopper-v4`*. Then, we moved to the* `PandaPush-v3` *manipulation task and trained Stable-Baselines3 PPO and SAC agents under source and target dynamics, where the target object is heavier than the source object. The final implementation includes an evaluation for fixed and randomized object mass and friction, a reward-shaping wrapper, Uniform Domain Randomization (UDR) over mass and friction, and an Automatic Domain Randomization (ADR) wrapper. The main results obtained for the PandaPush-v3 task show that a well tuned SAC policy can transfer quite well under low sim-to-real changes. However, when the sim-to-real gap is marked (hard evaluation setting) vanilla SAC fails and Domain Randomization (DR) are needed to close the gap.*

1. Introduction

Reinforcement learning (RL) formulates control as sequential decision making, where an agent learns a policy by interacting with an environment and maximizing expected return [9]. In robotics this setting is attractive because it can optimize behaviors that are difficult to hand-design, but it is also difficult because interactions are expensive, exploration can be unsafe, and the learned policy can exploit simulator-specific details [4, 5]. A common practical solution is to train in simulation and transfer the policy to the real robot. The central difficulty is the reality gap: the simulator dynamics are never exactly equal to the deployment dynamics [3].

This project follows the course sim-to-real abstraction in a controlled sim-to-sim benchmark. In Part 1, we implement basic policy-gradient algorithms on `Hopper-v4` to study the variance and stability issues behind direct policy optimization. In Part 2, we consider `PandaPush-v3`, where the source domain involves a light object and the target domain involves a heavier object. The target domain serves as a proxy for the real world: it is available for evaluation, but direct training on it is treated as an upper bound, rather than as a practical deployable solution.

The second part uses modern actor-critic algorithms from Stable-Baselines3. In particular, we consider PPO, which constrains policy updates through a clipped surrogate objective [8], and SAC, an off-policy maximum-entropy method that learns stochastic policies and reuses data through a replay buffer [2]. To reduce the transfer gap, we also implement domain randomization, which trains the agent over a distribution of simulator parameters rather than a single nominal model [6, 7, 10]. Specifically, the implemented UDR wrapper samples parameters from fixed ranges, while the ADR wrapper adapts the mass range based on recent success, following the curriculum idea behind automatic domain randomization [1]. These algorithms and randomization strategies will be discussed in more detail in the following chapters.

2. Methodology

2.1. Part 1: Policy Gradients on Hopper

The first part uses `Hopper-v4`, a continuous-control MuJoCo task. The action is continuous and the policy is stochastic. In `part1/agent.py`, the policy network is a two-hidden-layer MLP with 64 units per layer and `tanh` activations. The actor outputs the mean of a diagonal Gaussian distribution; the standard deviation is a learned parameter passed through `softplus`. All Part 1 experiments use Adam with learning rate 3×10^{-4} and discount factor $\gamma = 0.99$.

REINFORCE For an episode trajectory, the implementation computes Monte-Carlo returns

$$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k, \tag{1}$$

and minimizes the negative policy-gradient objective

$$\mathcal{L}_{\mathrm{PG}} = -\sum_t (G_t - b) \log \pi_\theta(a_t|s_t), \tag{2}$$

where b is either zero or a constant baseline.

Actor-Critic Variant. The actor-critic implementation uses the same actor network plus a critic network that predicts $V_\phi(s)$. At each update, the critic is trained by regression to the one-step bootstrapped target

$$y_t = r_t + \gamma V_\phi(s_{t+1})(1 - d_t), \tag{3}$$

where d_t indicates episode termination and the next-state value is detached from the gradient. The actor uses the corresponding advantage estimate

$$A_t = y_t - V_\phi(s_t), \tag{4}$$

which is normalized over the collected episode before computing the policy-gradient loss.

2.2. Part 2: PandaPush Pipeline

The second part uses the `panda-gym` environment. The source object mass is 1.0 kg, while evaluation is performed on two target settings that will be described in the evaluation protocol. The goal position is fixed at the center of the table, while the object initial position is randomized in the xy plane. The training scripts use the dense task reward and an episode horizon of 500 steps.

Algorithms. The second part relies on the Stable-Baselines3 implementations of two state-of-the-art actor-critic algorithms. PPO [8] is an **on-policy** method that collects rollouts under the current policy and updates it by maximizing a clipped surrogate objective,

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\big[\min\big(\rho_t A_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\, A_t\big)\big], \tag{5}$$

where $\rho_t = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio, A_t is a GAE advantage estimate, and the clip range ϵ bounds the policy update. SAC [2] is an **off-policy** maximum-entropy method that reuses past transitions from a replay buffer and optimizes the entropy-regularized objective

$$J(\pi) = \mathbb{E}\left[\sum_t r_t + \alpha\, \mathcal{H}\big(\pi(\cdot|s_t)\big)\right], \tag{6}$$

where $\mathcal{H}$ is the policy entropy and the temperature α (tuned automatically) trades off reward and exploration. Because the manipulation observation is a dictionary (achieved goal, desired goal, and state), both algorithms use the `MultiInputPolicy`.

Training Setting The training was performed trying different hyperparameters, and then choosing the ones that best performed. Table 1 reports the main hyperparameters used for the final SAC and PPO runs.

Hyperparameter	SAC	PPO
Environment	**source**	source
Sampling strategy	**none**	**none**
Training steps	**500k**	**500k**
Policy	**`MultiInputPolicy`**	**`MultiInputPolicy`**
Learning rate	**0.001**, $3e^{-4}$, 10^{-4}	10^{-3}, **2e-4**, $3e^{-4}$, 10^{-4}
Discount γ	**0.95**, 0.98, 0.99	**0.95**, 0.98, 0.99
Batch size	512, 1024, **2048**	**64**
Gradient steps / epochs	**-1**	**10**
Rollout steps	–	**2048**
Learning starts	**100k**	–
Replay buffer size	$\mathbf{10^6}$	–
Target update τ	**0.05**	–
Entropy coefficient	**auto**	**0.0**
GAE λ	–	**0.95**
Clip range	–	**0.2**
Policy network	**default SB3**	**[256, 256]**

Table 1. Main hyperparameters used for the Part 2 Stable-Baselines3 training runs. Dashes indicate algorithm-specific parameters that are not used. Bold values indicate the final parameters selected for the evaluation models.

Reward Shaping. The wrapper `part2/wrappers.py` changes only the scalar training reward, not the dynamics. The shaped reward is

$$r_{\text{train}} = r_{\text{env}} - p + B\mathbf{1}[d(o,g) < \epsilon], \tag{7}$$

where:
- p is a per-step penalty,
- B is a close-goal bonus,
- $d(o, g)$ is the object-goal distance

The values used for the training are $p = 10^{-2}$, $B = 1.0$, and $\epsilon = 0.05$.

Reward shaping was added on top of the dense environment reward to encourage faster goal reaching through a small per-step penalty and an additional bonus when the object is close to the target. Evaluation uses the environment reward directly, so success rate is not computed from the shaped reward.

Domain Randomization. The wrapper `part2/rand_wrapper.py` implements three modes. In `none`, the nominal environment dynamics are kept. In `udr`, the wrapper samples a new mass and friction values at every reset. The UDR training ranges are:

$$m \sim \mathcal{U}(1, 5), \quad \mu_{\text{obj}}, \mu_{\text{table}} \sim \mathcal{U}(0.5, 1.2), \tag{8}$$

and object spinning friction is sampled from $\mathcal{U}(0, 0.005)$. In `adr`, the wrapper starts from the nominal mass and friction and adapts the sampling ranges using a window of 20 episode outcomes. If the mean success rate is at least 0.70, the mass upper bound expands toward 5 kg and the friction ranges widen toward their global limits. If the mean success rate is at most 0.35, the ranges contract back toward the nominal values. With the default step size, the mass range

expands by 1 kg per successful update, while lateral-friction ranges expand by 0.175 per update.

Evaluation Protocol. Each model is evaluated with deterministic actions for 50 episodes, using seeds 0, 42, and 99. The main metric is the final success rate, computed from `info["is_success"]` at the end of each episode. We use two target settings: an easy fixed target with the nominal target mass, and a hard target where mass and friction are randomized at test time, from a uniform distribution. We took this direction in order to test the model in various and unknow settings. This was done to achieve a single policy capable of generalizing different materials (both for the surface and the object itself) as well as different mass.

Setting	Mass (kg)	Object μ	Table μ	Object spin
Easy target	5.0	0.5	0.5	0.0
Hard target	$\mathcal{U}(5, 10)$	$\mathcal{U}(0.2, 1.5)$	$\mathcal{U}(0.2, 1.5)$	$\mathcal{U}(0, 0.01)$

Table 2. Target-domain evaluation settings used for the final success-rate comparison.

3. Experimental Results

3.1. Part 1: Hopper

Figure 1 compares REINFORCE without baseline, REINFORCE with a constant baseline of 20, and the actor-critic variant over seeds 0, 42, and 99. All methods improve from the initial policy, but the curves remain noisy, as expected for policy-gradient training on Hopper. The constant baseline gives the strongest final training curve and reduces some early variance relative to plain REINFORCE. The actor-critic variant learns more slowly and reaches lower average returns in this implementation.

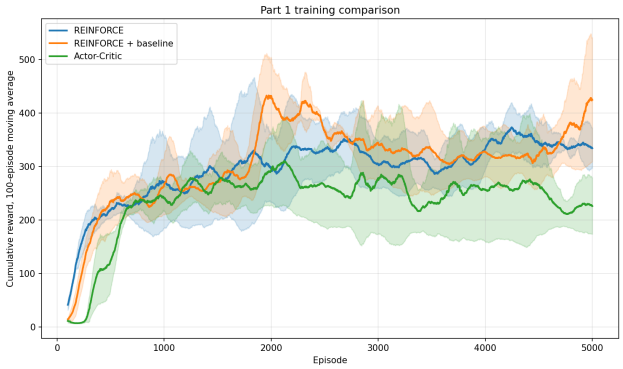

Figure 1. Part 1 training comparison on `Hopper-v4`. Curves show the 100-episode moving average of cumulative reward, averaged over seeds 0, 42, and 99; shaded regions show variability across seeds.

3.2. Part 2: PandaPush-v3

For the second part of the project both PPO and SAC models were trained, however since the training of PPO was more unstable and it does not converge, only SAC was kept for further implemantations and evaluations.

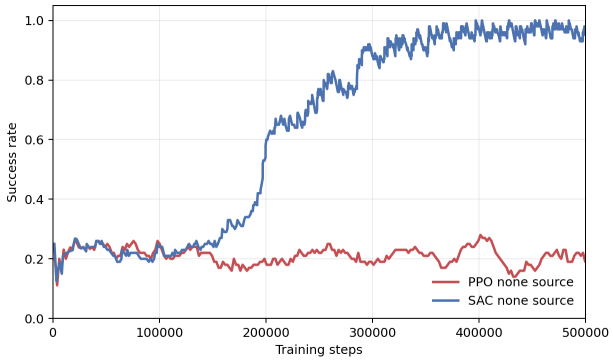

Figure 2. Training success-rate comparison on the source-domain `PandaPush-v3` task without domain randomization. The PPO curve corresponds to `PPO_none_source_500k_lr2e-4_g095_1`, while the SAC curve corresponds to `SAC_none_source_500k_2`.

3.3. Nominal Sim-to-Sim Evaluation

The first section of table 3 reports aggregated 150-episodes evaluations (50 episodes per seed) generated from the saved SAC checkpoints. At the nominal target mass of 5 kg, the source-trained SAC policy already transfers well: the gap between source to source and source to target is small when analyzing success rate.

3.4. Harder Mass and Friction Evaluation

Because nominal mass transfer was already strong, the evaluation was made harder by randomizing parameters at test time. Table 3 uses mass sampled uniformly in $[5, 10]$ kg, object/table lateral friction sampled in $[0.2, 1.5]$, and object spinning friction sampled in $[0, 0.01]$. Under this harder protocol, the non-randomized source policy drops from 97.3% to 72% success on target. This result was used as Lower Bound for further experiments using domain randomization.

A stronger target-distribution baseline was then obtained by training UDR directly on the target-domain intervals, with mass sampled from $\mathcal{U}(5, 10)$ kg and lateral friction from $\mathcal{U}(0.2, 1.5)$, matching the hard evaluation distribution exactly. This model reaches 85.3% success on the hard target over the same aggregated 150 episodes, improving over the non-randomized lower bound but remaining below the source-domain UDR and ADR policies.

Model	Evaluation	Mean return	Std return	Success
SAC source	target (easy)	-1.740	7.917	97.3%
SAC target	target (easy)	-0.608	2.237	99.3%
SAC source (**Lower Bound**)	target (hard)	-12.762	19.722	72.0%
SAC UDR hard target	target (hard)	-6.970	16.423	85.3%
SAC UDR + friction source	target (hard)	-4.814	14.555	90.7%
SAC ADR + friction source	target (hard)	-4.638	27.440	94.0%

Table 3. Deterministic SAC evaluations aggregated over 150 episodes, using 50 episodes for each seed in $\{0, 42, 99\}$. The hard target samples mass from $\mathcal{U}(5, 10)$ kg, object/table lateral friction from $\mathcal{U}(0.2, 1.5)$, and object spinning friction from $\mathcal{U}(0, 0.01)$.

3.5. Domain Randomization: UDR and ADR

In order to close the reality gap under hard evaluation, UDR and ADR policies were trained and tested. The UDR policy trained with mass and friction randomization reaches 90.7%. Moreover, the ADR policy generalizes even better on unseen distribution, achieving 94% of SR on the hard evaluation setting. Although direct UDR training on the target-domain distribution is an intuitive strong baseline, both source-domain UDR and ADR outperform it under the current training budget. Despite seeing the same distribution during training and evaluation, the UDR hard target model still performs worse than the ADR curriculum model. The per-bin breakdown in Table 4 uses the same 150 hard-target evaluation episodes as Table 3; ADR is stronger in every object-lateral-friction bin, and the largest gap appears at high table friction.

μ bin	N	UDR hard SR	ADR SR
Object lateral friction			
[0.20, 0.46)	33	90.9%	93.9%
[0.46, 0.72)	19	84.2%	89.5%
[0.72, 0.98)	33	78.8%	87.9%
[0.98, 1.24)	41	82.9%	100.0%
[1.24, 1.50]	24	91.7%	95.8%
Table lateral friction			
[0.20, 0.46)	25	92.0%	96.0%
[0.46, 0.72)	33	90.9%	97.0%
[0.72, 0.98)	38	86.8%	94.7%
[0.98, 1.24)	20	90.0%	90.0%
[1.24, 1.50]	34	70.6%	91.2%
Overall	150	85.3%	94.0%

Table 4. Friction-bin success rates for `SAC trained on the hard target (UDR using full distribution)` and `SAC using ADR` on the hard target deterministic evaluation, aggregated over 150 episodes using 50 episodes for each seed in $\{0, 42, 99\}$. The hard target uses the same mass and friction randomization protocol as Table 3.

4. Discussion

4.1. Result Analysis

For Part 1, the Hopper experiments show the expected behavior of simple policy-gradient methods. REINFORCE learns but remains noisy because Monte-Carlo returns have high variance, while the constant baseline reduces this variance and gives the strongest training curve among the implemented methods. The actor-critic variant learns more slowly in this setup, suggesting that the bootstrapped critic would require more tuning to outperform the simpler baseline correction.

For Part 2, the most important observation is that the nominal source to target transfer is easier than expected from mass alone. A source-trained SAC policy reaches 97.3% success on the 5 kg target object, only slightly below the 99.3% target-trained policy. This suggests that the current `PandaPush-v3` setup is not strongly affected by the fixed 1 kg to 5 kg mass shift. The task has a fixed goal, a short object-goal distance distribution, dense rewards during training, and a powerful continuous-control algorithm. These factors can hide part of the intended reality gap. The harder setting was introduced to obtain results that are more meaningful and applicable to real-world scenarios. When target mass is sampled in $[5, 10]$ kg and friction is randomized, the non-randomized SAC policy falls to 72.0% success. This indicates that friction and heavier masses expose failures that are not visible in the nominal target evaluation. Observing the result of the training, SAC has better performance compared to PPO. The experiments show that SAC achieves training success after 500k steps, while PPO runs are less consistent. This is coherent with the algorithms: SAC is off-policy and can reuse data through its replay buffer, whereas PPO is on-policy and discards rollouts after each update. For goal-conditioned pushing, replay-based learning is especially useful because informative transitions can be reused many times.

4.2. Analysis of ADR and UDR

After introducing the hard target setting, we trained a UDR model directly on the hard target distribution as a strong

target-distribution baseline. This model achieved a success rate of 85.3%. At first, this imperfect result seemed to be a direct consequence of the increased difficulty of the task. However, the source-domain UDR and ADR policies outperformed it, reaching 90.7% and 94.0% success respectively.

A natural first hypothesis is that some combinations of mass and friction are physically difficult for the robot. The Panda joints have limited torques (87 N on most joints), and pushing a 10 kg object under high friction may be infeasible or close to infeasible for some sampled configurations. This can slow down learning and make convergence less stable when the policy is exposed to the full hard distribution from the beginning. However, this explanation is not sufficient on its own: ADR is evaluated on the same hard target distribution and still reaches 94.0% success. Therefore, the gap is more likely related to learning efficiency than to physical infeasibility alone. The source-domain UDR policy is trained on an easier distribution, with lower masses and a narrower friction range. This can produce faster and more stable convergence, because the policy is not exposed to the hardest dynamics from the start. ADR strengthens this effect through a curriculum: it expands the sampling range only when the current policy reaches a success rate above 70%. As a result, the policy is not overwhelmed by the full target difficulty at the beginning of training. It first learns reliable pushing behavior in easier conditions, then gradually adapts to heavier objects and more variable friction. By the time the model reaches the harder end of the range, it has already acquired a useful behavioral prior that can be refined instead of learning a robust strategy from scratch.

4.3. Reward Shaping

The reward-shaping wrapper affects training but not the evaluation success metric. The time penalty encourages short solutions, and the close-goal bonus increases the learning signal once the object reaches the success threshold. For SAC, this can increase success rate indirectly by making successful behavior easier to learn early in training. However, it also makes training return less comparable to raw evaluation return. This is why the evaluation script reports both return and success rate on the unwrapped environment.

4.4. Methodological Critique

The main limitation of the experimental setup is that the reported evaluations are averaged over three evaluation seeds, but each policy comes from a single training run. Therefore, the results are more reliable than a single 50-episode evaluation, but they do not capture the variance due to different training seeds. Stronger conclusions would require training each configuration multiple times.

A second limitation is that both UDR and ADR depend strongly on the chosen randomization ranges and hyperparameters. If the UDR range is too wide, learning can become unstable or inefficient; if it is too narrow, the policy may not become robust enough. Similarly, ADR depends on the success threshold and expansion rule used for the curriculum. In particular, the ADR step is relatively coarse. Although this worked well empirically, a smaller step could provide a smoother curriculum and reduce sensitivity to transient success-rate estimates.

Finally, this project studies transfer in a simplified sim-to-sim setting. Although changing mass and friction exposes a meaningful reality gap, it does not include all sources of mismatch that appear in real robotics, such as sensor noise, actuator delays, inaccurate contacts, or calibration errors. For this reason, the results should be interpreted as evidence that domain randomization improves robustness in simulation, not as a guarantee of real-world transfer.

5. Conclusion

This project implemented policy-gradient methods on Hopper and Stable-Baselines3 policies for sim-to-sim transfer on PandaPush. In Part 1, the REINFORCE variant with a constant baseline gave the strongest training curve among the implemented methods, while the actor- critic version required more tuning. In Part 2, SAC was the most reliable algorithm, and the nominal source-to-target transfer was easier than expected, reaching 97.3% success on the target domain.

The harder randomized evaluation showed a clearer reality gap: the non-randomized source policy dropped to 72.0% success when mass and friction were randomized. Domain randomization improved robustness, with UDR reaching 90.7% and ADR achieving the best result at 94.0%. The main lesson is that curriculum-based randomization can be more data-efficient than exposing the policy to the full hard distribution from the beginning. The main limitation is that each policy was trained only once; future work should use multiple training seeds and test a wider set of simulator parameters.

6. Acknowledgments

6.1. Declaration of not use

The authors of this work declare that Generative AI tools were not used to write the project report.

6.2. GenAI for the code

Generative AI tools were used to support the development of the project code as follows:

- **Name and Version**: Codex 5.5
- **Publisher**: OpenAI
- **Description**: Codex was used for the following tasks: refactoring the repository; adding detailed `README.md`, `commands.md`, and `help.md` files to improve experiment reproducibility; adding command-line interfaces for smoother testing, argument parsing, and meaningful debug printing; and writing plotting code.
- **Name and Version**: Gemini 3.1 Pro
- **Publisher**: Google
- **Description**: Gemini was used mainly for deep research on relevant material to select hyperparameters before running experiments, in order to avoid long tuning runs. The search output and the provided sources were reviewed.

The authors further declare that the overall logic and planning were not produced using AI.

References

[1] Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving rubik's cube with a robot hand. *arXiv preprint arXiv:1910.07113*, 2019. 1

[2] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870, 2018. 1, 2

[3] Sebastian Hofer, Kostas Bekris, Ankur Handa, Juan Camilo Gamboa, Melissa Mozifian, Florian Golemo, et al. Perspectives on sim2real transfer for robotics: A summary of the r:ss 2020 workshop. *arXiv preprint arXiv:2012.03806*, 2020. 1

[4] Jens Kober, J. Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013. 1

[5] Petar Kormushev, Sylvain Calinon, and Darwin G. Caldwell. Reinforcement learning in robotics: Applications and real-world challenges. *Robotics*, 2(3):122–148, 2013. 1

[6] Fabio Muratore, Tim Gruner, Florian Wiese, Boris Belousov, Michael Gienger, and Jan Peters. Robot learning from randomized simulations: A review. *Frontiers in Robotics and AI*, 9, 2022. 1

[7] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation*, pages 3803–3810, 2018. 1

[8] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 1, 2

[9] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, 2018. 1

[10] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. *arXiv preprint arXiv:1703.06907*, 2017. 1